

Residency Day 3: Group Project: Deliverable 3

An Phan and Vatsalkumar Mukeshkumar Dholakiya (Group 9)

University of the Cumberland

MSCS 632 - Advanced Programming Languages

Dr. Jay Thom

October 4, 2026

We built a collaborative to-do application with one React frontend and two interchangeable backends: Java with Javalin and JavaScript with Node.js and Express. Both use the same REST API and JSON structure, allowing a direct comparison of the two languages without changing the frontend. This report compares their syntax, language features, concurrency, tooling, developer experience, and performance, and examines how each language influenced backend design.

Programming Paradigms

The two languages led to different structures for the same logic. Java is statically typed and class-based, so we used an object-oriented design with classes such as Task, User, and Comment, plus separate TaskRepository, TaskService, and Main components. Its compiler enforces object types and catches mismatches early, making the structure clear and self-documenting, but also more verbose. JavaScript is dynamically typed and multi-paradigm, so the Node backend used a lighter function-and-module approach. We used factory functions such as createStore, createService, and createApp, with plain JavaScript objects instead of formal classes. This made development faster and more flexible, but the structure depended more on convention because it was not enforced by the compiler.

The same operation illustrates the contrast. Creating a task in Java:

Java

```
Task t = new Task();
t.id = UUID.randomUUID().toString();
t.title = input.title;
t.status = "Open";
t.createdAt = Instant.now().toString();
return repo.add(t);
```

java

and in JavaScript:

JavaScript

```
const task = {
  id: randomUUID(),
  title: input.title,
  status: "Open",
  createdAt: new Date().toISOString(),
};
tasks().push(task);
await store.persist();
return task;
```

Both produce the same JSON, but Java states the type of everything while JavaScript simply builds the object.

Concurrency Models

The requirement to edit tasks concurrently by multiple users exposed the biggest difference between the two runtimes. Java uses real operating-system threads. Javalin serves each incoming request on its own thread from a pool, so two requests can genuinely run in parallel on multiple CPU cores. Parallelism is powerful but dangerous for shared mutable state: if two threads read-modify-write the same task at the same instant, one update can be lost, or the data file can be corrupted mid-write. We solved this the idiomatic Java way, with explicit locking. The repository wraps its task map in a `ReentrantReadWriteLock`: reads take a shared read lock (many can proceed at once), while writes take an exclusive write lock and persist to disk inside it, so the entire read-modify-write-save cycle is atomic:

Java

```
public Task update(String id, Consumer<Task> mutator) {
    lock.writeLock().lock();
```

```

    try {
        Task t = tasks.get(id);
        if (t == null) return null;
        mutator.accept(t);
        persistUnlocked(); // write happens while the lock is
held
        return t;
    } finally { lock.writeLock().unlock(); }
}

```

Node.js takes the opposite approach: a single thread with an event loop. There is no parallel execution of our JavaScript, so a synchronous block of code - such as mutating an in-memory object - is inherently atomic; nothing else runs until it finishes or hits an "await". The danger is not parallel memory access but interleaved asynchronous I/O: two overlapping requests could both start writing tasks.json and clobber the file. We solved this not with locks (there are no threads to lock) but by serializing the writes through a promise queue, so each disk write waits for the previous one:

```

JavaScript
function persist() {
    const write = persistQueue
        .catch(() => {})
        .then(() => writeFile(filePath, JSON.stringify(state, null,
2))));
    persistQueue = write.catch(() => {});
    return write;
}

```

The result is the same guarantee - the shared store never corrupts under concurrent edits - reached by opposite means: Java coordinates multiple threads with a lock; Node avoids threads

entirely and orders its asynchronous work. One honest limitation we kept identical in both is that field updates are "last-write-wins." The lock and the queue prevent corruption, but neither prevents a lost update when two users edit the same field at once; preventing that would require optimistic locking (version numbers). To show safe concurrency without that problem, comments are append-only in both backends, so concurrent comments never overwrite each other.

Language Features and Syntax

Java and JavaScript each had clear advantages. Java's static typing caught mistakes at compile time and provided more structure, but required more code and setup. JavaScript was faster to develop with, handled JSON naturally, and made asynchronous I/O simple through promises and `async/await`, though it relied more on testing and coding conventions for correctness. Overall, Java was more structured and maintainable for larger systems, while JavaScript was more concise and convenient for a small JSON-based service.

Performance

For this application, the dataset is small, so both backends feel instantaneous, and performance differences are mostly theoretical rather than noticeable in practice. Java's multithreaded model can use multiple CPU cores, making it better suited for CPU-intensive tasks or very high workloads, though it introduces additional thread and locking overhead. The JVM also requires some warm-up time before reaching peak performance. Node.js uses a single-threaded event loop that works especially well for I/O-heavy applications with many concurrent connections. However, CPU-intensive tasks can block the event loop and delay other requests. Since our application primarily performs small JSON file reads and writes via an HTTP

API, Node.js fits the workload well, while Java may offer greater scalability for heavier multi-core workloads.

Developer Experience and How Each Language Shaped Our Approach

JavaScript felt faster and more flexible because it required less boilerplate, handled JSON naturally, and supported async operations directly. However, correctness depended more on testing and developer discipline, which can become risky as a project grows. On the other hand, Java felt more structured and safer because its compiler and type system caught errors early and clarified the architecture. The trade-off was greater verbosity and a heavier development setup. Each language also influenced the backend design. Java's multithreaded model required explicit locking for shared state, while Node.js avoided locks and instead required careful ordering of asynchronous file writes. Rather than forcing the same internal design on both, we used the approach that best matched each language while keeping the external API behavior identical.

Conclusion

Building the same application in Java and JavaScript showed that each language encourages a different development model. Java offered stronger structure, static typing, and multithreading, while JavaScript provided simpler syntax, native JSON, and efficient asynchronous I/O. Both met the same requirements, but used different concurrency approaches: Java with locks and Node.js with an async write queue. The main lesson is that the best design depends on the strengths of the language being used.